

Credit Scoring Based on Card Transaction Data

*Behavioral modeling, proxy target
construction, and model comparison*

Report : Smirnova, Artamonov, Verevkin



CONTENTS

01 Problem & Motivation

03 Exploratory Data
Analysis

05 Proxy Target
Construction

07 Model Comparison

02 Data Description &
Quality

04 Feature Engineering &
Account Profiling

06 Modeling Approaches

08 Key Findings &
Conclusion

PART. 01

PROBLEM & CONTEXT

Problem & Motivation

In real-world financial datasets, explicit fraud or default labels are often unavailable or delayed.

However, financial institutions still need to identify risky or suspicious accounts based on behavioral signals.

Goal of the project:

To develop a scoring framework that identifies potentially suspicious accounts using transaction data **without a true target variable**.





PART. 02

Dataset Overview

Data Description

The raw data includes transaction timestamps, transaction amounts, account balances, login attempts, device identifiers, IP addresses, locations, and merchants.



All date variables were converted to datetime format, and numerical variables were cast to numeric types to ensure consistency for further analysis.

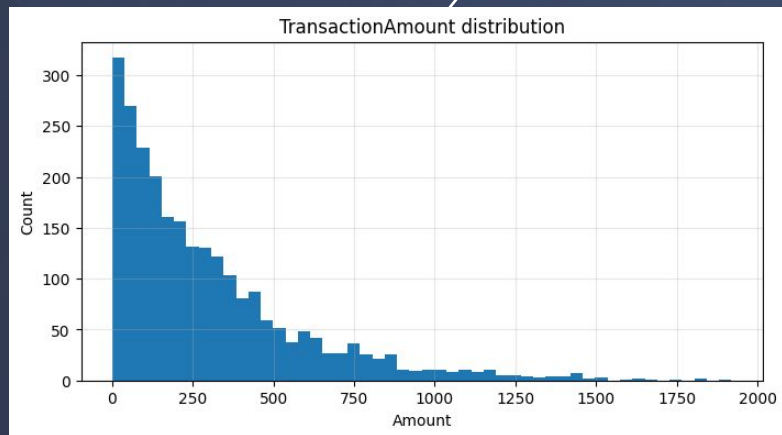
	dtype	missing_%	n_unique
TransactionID	object	0.0	2512
AccountID	object	0.0	495
TransactionAmount	float64	0.0	2455
TransactionDate	datetime64[ns]	0.0	2512
TransactionType	object	0.0	2
Location	object	0.0	43
DeviceID	object	0.0	681
IP Address	object	0.0	592
MerchantID	object	0.0	100
Channel	object	0.0	3
CustomerAge	int64	0.0	63
CustomerOccupation	object	0.0	4
TransactionDuration	int64	0.0	288
LoginAttempts	int64	0.0	5
AccountBalance	float64	0.0	2510
PreviousTransactionDate	datetime64[ns]	0.0	360

PART. 03

EXPLORATORY DATA ANALYSIS

Transaction amount distribution

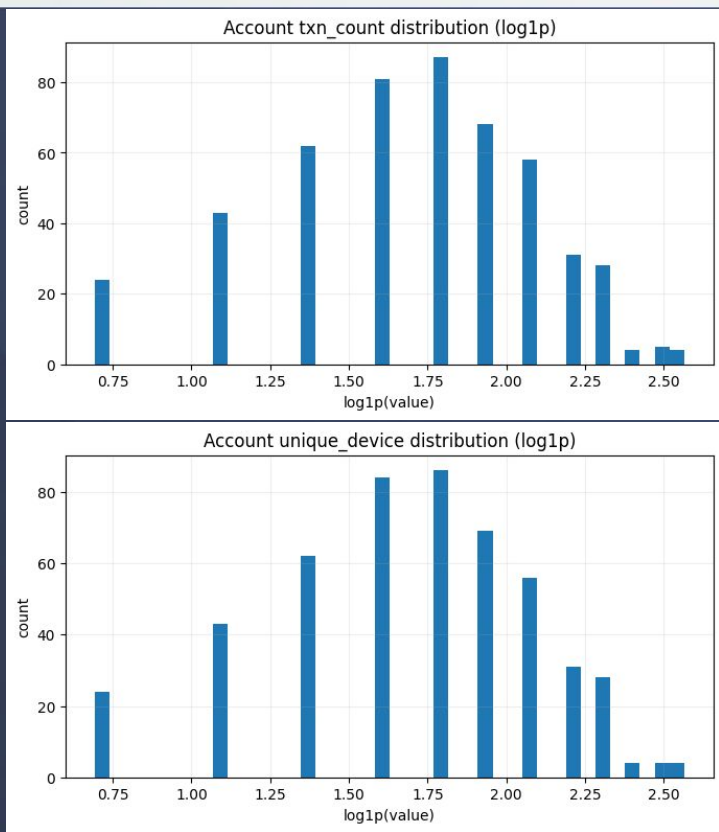
Transaction amounts show a right-skewed distribution, which is typical for real transactional data. This confirms the feature looks realistic and motivates robust aggregation at account level.



Behavioral Patterns at Account Level

After aggregation to the account level, strong heterogeneity in customer behavior becomes visible.

A small number of accounts demonstrate significantly higher activity and technical variability.



PART. 04

**Feature Engineering and
Account Profiling**

Engineered features example

Feature engineering focuses on capturing abnormal behavior across multiple dimensions:

monetary intensity,
transaction frequency,
temporal irregularity,
technical instability.

Both transaction-level and account-level features are used.

```
1 df["TimeSincePrevTransaction"] = (  
    df["TransactionDate"] - df["PreviousTransactionDate"]).dt.total_seconds()  
df["TimeSincePrevTransaction"] = df["TimeSincePrevTransaction"].fillna(0)
```

```
1 df["AmountToBalanceRatio"] = df["TransactionAmount"] / (df["AccountBalance"] + 1)
```

Account-level aggregation overview

Transactions were aggregated to account level to build a behavioral profile per AccountID. The profile includes spending statistics, activity counts, time regularity, and diversity measures

	AccountID	mean_amount	median_amount	std_amount	max_amount	min_amount	avg_balance	txn_count	debit_count
0	AC00001	130.380000	130.38	116.799898	212.97	47.79	2915.160000	2	2
1	AC00002	293.744286	331.66	195.695091	516.47	12.62	4480.421429	7	6
2	AC00003	253.268000	237.97	158.484753	416.62	20.17	7329.568000	5	4
3	AC00004	242.231111	136.31	231.081586	642.54	13.16	3340.956667	9	8
4	AC00005	347.974444	377.92	189.487268	680.76	107.48	4457.837778	9	8

PART. 05

PROXY TARGET CONSTRUCTION

Proxy target creation

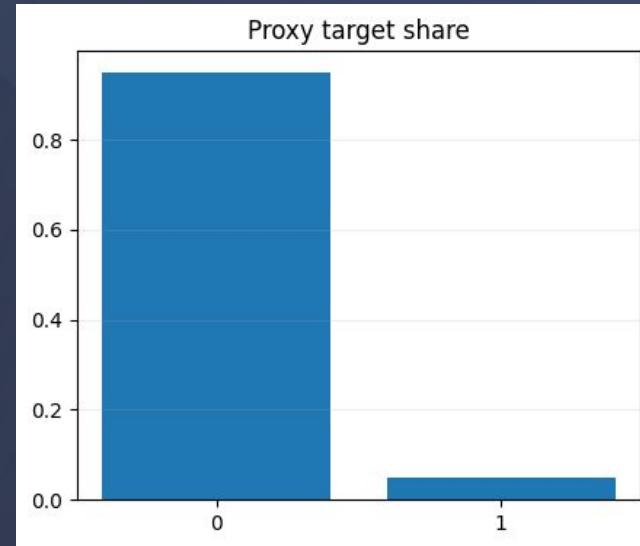
Because no labels exist, we constructed a proxy target via unsupervised anomaly detection. Isolation Forest was trained on numerical account features with contamination set to 5%, producing an anomaly flag used as the final target.

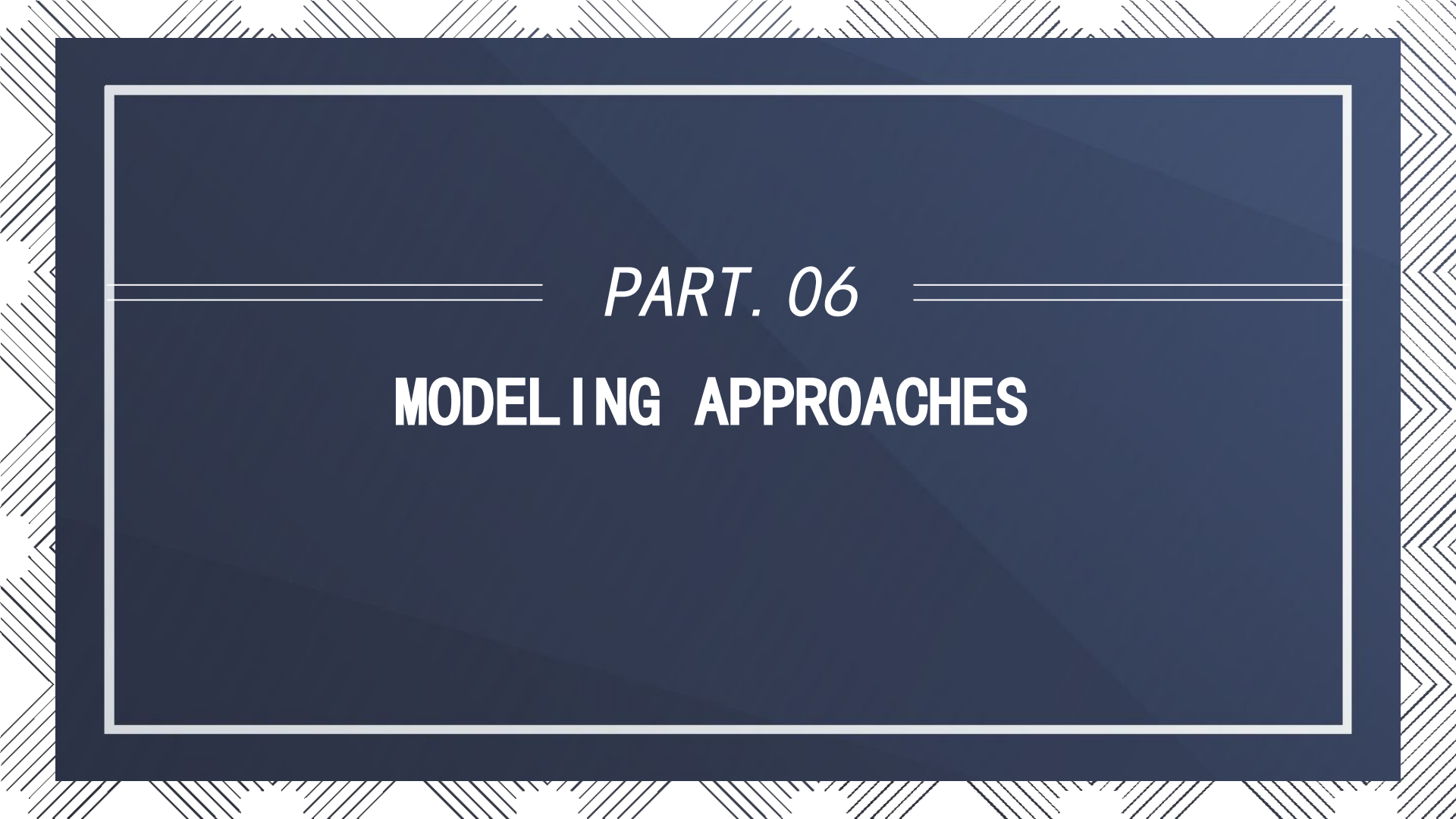
```
cont = 0.05
iso = IsolationForest(
    n_estimators=600,
    contamination=cont,
    random_state=42
)
iso.fit(X_prep)
```

Доля аномалий IsolationForest: 0.050505050505050504
Доля аномалий LOF: 0.050505050505050504
доля target=1: 0.050505050505050504

Target rate

The proxy target prevalence is approximately 5%, aligned with a realistic assumption for suspicious behavior. This provides a workable signal for supervised modeling and comparison.





PART. 06

MODELING APPROACHES

Baseline Model: Logistic Regression with WOE

Interpretable baseline model

Numerical account-level features are binned into deciles and transformed using **Weight of Evidence (WOE)**.

Information Value (IV) is used to assess feature predictive power.

A **logistic regression** model is trained on WOE-transformed features.

Test performance:

ROC-AUC = 0.92 , **KS = 0.77**

This approach provides strong performance while remaining fully interpretable and aligned with classical credit scoring.

	feature	coef
13	std_time_between_woe	1.648475e-01
6	txn_count_woe	1.156365e-01
15	unique_devices_woe	8.108346e-02
9	night_txn_ratio_woe	-2.007327e-08
14	unique_ip_woe	-3.753769e-03
12	avg_time_between_woe	-1.719462e-02
2	std_amount_woe	-1.676446e-01
7	debit_count_woe	-1.956696e-01
17	unique_merchants_woe	-2.017253e-01
5	avg_balance_woe	-2.353373e-01
18	avg_login_attempts_woe	-3.464523e-01
4	min_amount_woe	-3.652958e-01
3	max_amount_woe	-3.817291e-01
10	debit_ratio_woe	-4.613114e-01
0	mean_amount_woe	-5.765122e-01
1	median_amount_woe	-5.931366e-01
8	credit_count_woe	-9.441007e-01
16	unique_locations_woe	-9.631681e-01
11	credit_ratio_woe	-1.056256e+00

	feature	IV
16	unique_locations	8.792012e+00
14	unique_ip	8.412523e+00
17	unique_merchants	8.380203e+00
15	unique_devices	8.337416e+00
6	txn_count	8.297500e+00
7	debit_count	7.924586e+00
3	max_amount	5.421108e+00
0	mean_amount	4.193137e+00
4	min_amount	3.980293e+00
2	std_amount	3.784659e+00
1	median_amount	3.079636e+00
5	avg_balance	2.850828e+00
10	debit_ratio	2.720810e+00
13	std_time_between	2.633127e+00
11	credit_ratio	2.222914e+00
12	avg_time_between	1.373153e+00
8	credit_count	5.914946e-01
18	avg_login_attempts	4.046941e-02
9	night_txn_ratio	3.111853e-15

Alternative Models



CatBoost



XGBoost



Random Forest



PART. 07

MODEL COMPARISON

Model Performance Comparison

Boosting and ensemble models significantly outperform logistic regression in terms of ROC-AUC and KS. However, this improvement comes at the cost of interpretability.

	Model	ROC-AUC (test)	KS (test)	Interpretability
0	Logistic Regression (WOE)	0.9220	0.7686	High (WOE, coefficients)
1	CatBoost	0.9982	0.9929	Medium
2	XGBoost	0.9965	0.9858	Medium
3	Random Forest	0.9947	0.9929	Medium



PART. 08

CONCLUSIONS



Key Findings

Account-level behavioral features are highly informative for detecting suspicious activity.
Logistic regression with WOE provides a strong and interpretable baseline.
Boosting and ensemble models achieve superior performance but reduce transparency.



Final Conclusion

This project demonstrates a practical framework for risk scoring in the absence of labeled data.
By combining anomaly detection, behavioral aggregation, and model comparison, it highlights the trade-off between interpretability and predictive performance.

